



UNIVERSIDADE FEDERAL DO CEARÁ

Campus de Quixadá

Prof. Bruno Góis Mateus

QXD0276- Desenvolvimento de para dispositivos móveis

AP2

2026.1

Nome: Guilherme Waleg Brito Feijões Matrícula: 582110

[1 ponto] 1. Considere uma aplicação que utiliza Jetpack Compose e segue a arquitetura recomendada pelo Google. Quando um usuário pressiona um botão para carregar uma lista de produtos, qual fluxo representa corretamente a comunicação entre os componentes?

A. UI → Banco de Dados → ViewModel

B. ViewModel → UI → Repository

☒ C. UI → ViewModel → Repository → Fonte de Dados

D. Repository → UI → Fonte de Dados

E. UI → Fonte de Dados → Repository

[1 ponto] 2. Considere o ViewModel abaixo:

```
class CounterViewModel : ViewModel() {
```

```
    private val _count = MutableStateFlow(0)
```

```
    val count = _count.asStateFlow()
```

```
    fun increment() {  
        _count.value++  
    }
```

```
}
```

Complete a linha faltante para expor o estado sem permitir modificações externas. ✓

[2 pontos] 3. Explique o papel do ViewModel na arquitetura de aplicativos recomendada pelo Google.

Em sua resposta, mencione: ✓

- sua principal responsabilidade;
- sua relação com a interface gráfica;
- uma vantagem em relação ao uso exclusivo da Activity para armazenar estado.

[2 pontos] 4. Uma aplicação obtém produtos de uma API REST e também os salva localmente utilizando Room.

Em qual camada da arquitetura você implementaria a lógica responsável por decidir se os dados devem vir da API ou do banco local?

Justifique sua resposta.

→ Data layer

[1 ponto] 5. Sobre Coroutines, marque V para verdadeiro e F para falso. ✓

OBS: Acerto = (+0,25pt). Erro = (-0,125pt). Pontos negativos não se propagam para as outras questões.

☒ (V) Uma coroutine pode ser suspensa e retomada posteriormente. ✓

☒ (F) Toda coroutine é executada em uma Thread diferente.

☒ (F) delay() bloqueia a Thread durante o tempo de espera.

☒ (F) launch retorna um valor para quem o chamou.

→ SO gerencia

Dispatchers → Main, IO, pa

launch retorna um Job

async retorna um Deferred<T>

Nota:

10,0

Parabéns!



[1 ponto] 6. Sobre StateFlow, marque V para verdadeiro e F para falso.

OBS: Acerto = (+0,25pt). Erro = (-0,125pt). Pontos negativos não se propagam para as outras questões.

☒ Possui um valor atual acessível pela propriedade value.

☒ Continua existindo mesmo sem coletores.

☒ É um tipo de Flow.

☒ Não pode ser observado pela UI.

→ com collect.

Precisa do collect para ser acessível
Flow para State Flow
Cold Stream Hot Stream

[2 pontos] 7. Um aplicativo exibe a cotação do dólar em tempo real.

Novos valores chegam continuamente do servidor.

Qual abordagem você utilizaria?

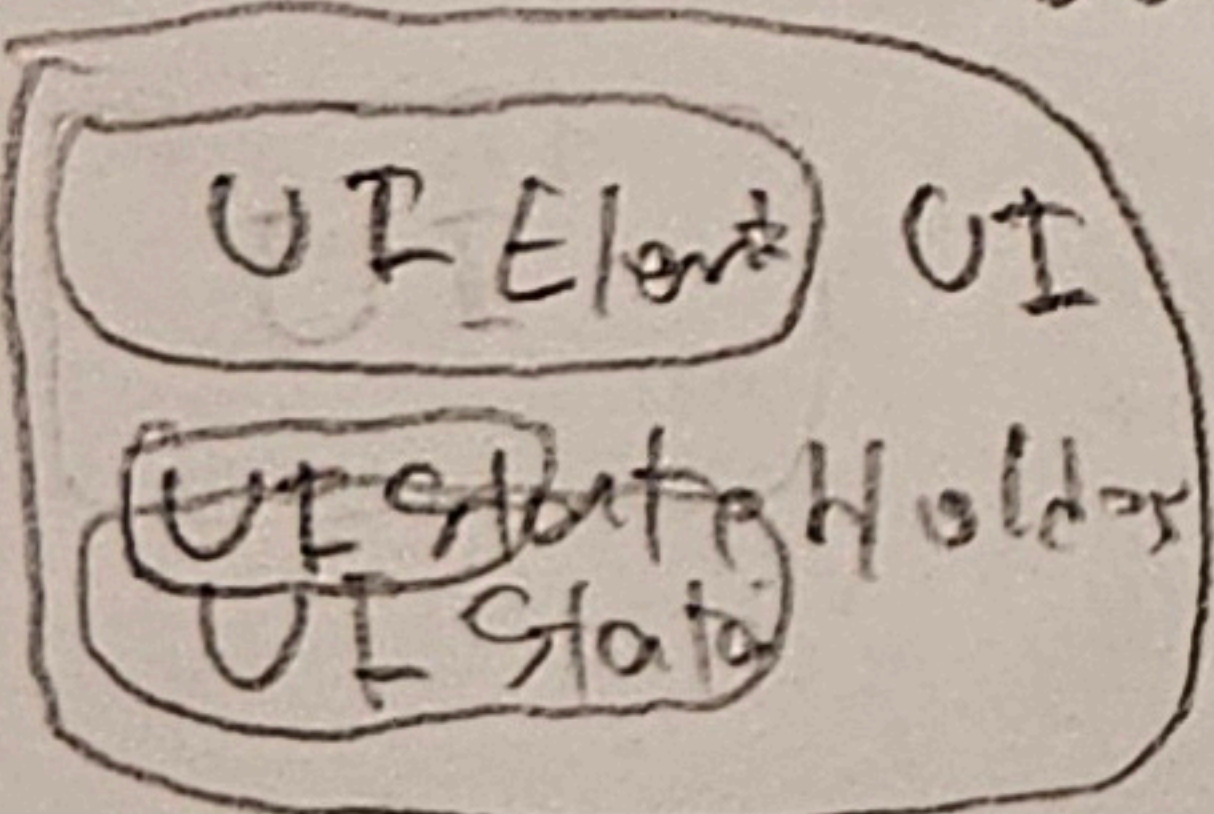
- suspend function
- Flow
- StateFlow

2+2+1+1+1

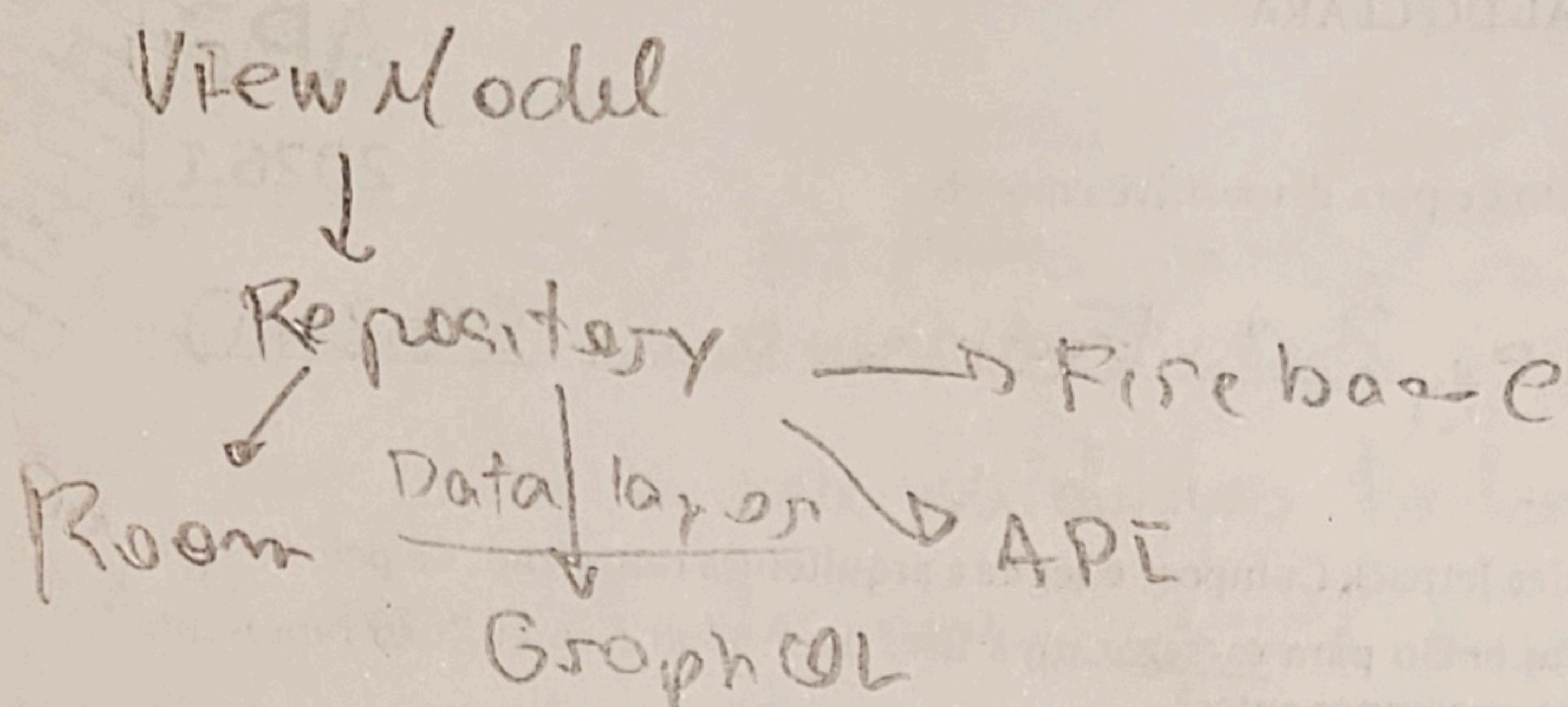
Justifique sua resposta.

Question	Points	Score
1	1	1
2	1	1
3	2	2
4	2	2
5	1	1
6	1	1
7	2	2
Total:	10	10

③ Principal funcionalidade do ViewModel é armazenar a lógica do UI. Ao invés de montar essa lógica de negócios no Activity ou Componente, essa responsabilidade é delegada para o View Model, e a UI apenas coleta o seu estado para usar nos UI's elements, a UI recebe o View Model direto no construtor. O View Model recebe a UI State que deve ser IMUTÁVEL, e cria as funções necessárias para a lógica do Interface, expondo o estado dele, a UI por sua vez coleta o estado dos elementos e se precisar alterar o estado, chama alguma função do VM para fazer isso. Vantagens do uso do VM ao invés de montar o estado no Activity, é que o VM não morre no ciclo de vida, mantendo o estado mesmo em uma rotação, por exemplo, também ficou mais fácil de testar e escalar a aplicação.



④ Essa responsabilidade é do repository, escolher de onde está vindo os dados.



Essa representação é a correta para a arquitetura atual, com offline-first. O View Model recebe apenas o repository e este por sua vez abstrai funcionalidades de onde está vindo, como está implementado etc. No qual o repository usa as DA para implementar a lógica.

6- User Repository (only)

① Query (SELECT * FROM usuarios)
fun getAll(): Flow<User>

② Insert/Update

suspend fun createUser()

7) Usaria um Flow pois ele é Cold Stream, isso tem uma grande vantagem sobre o StateFlow e suspend function nesse cenário que os dados estão sempre atualizando seu se fosse de tabelas, Suspend function seria melhor se fosse apenas um dado que fosse buscado ou que não mudasse constantemente. A característica que explica essa vantagem é que ele não busca os dados toda de uma vez, o flow retorna um estado observável e coleta ele, fazendo a busca quando for realmente necessário, não bloqueando a thread

Ex - StateFlow x Flow

```
objetos = ... .getAll()
for (objeto in objetos)
    println(objeto)
```

